

METAGPT: 多智能体协作框架的元编程

Sirui Hong^{1*}, Mingchen Zhuge^{2*}, Jiaqi Chen¹, Xiawu Zheng³, Yuheng Cheng⁴, Ceyao Zhang⁴, Jinlin Wang¹, Zili Wang¹, Steven Ka Shing Yau⁵, Zijuan Lin⁴, Liyang Zhou⁶, Chenyu Ran¹, Lingfeng Xiao^{1,7}, Chenglin Wu^{1†}, J. Jürgen Schmidhuber^{2,8}

DeepWisdom, AI Initiative, King Abdullah University of Science and Technology, 厦门大学, 香港中文大学 (深圳), 南京大学, 宾夕法尼亚大学, 加州大学伯克利分校, 瑞士AI实验室IDSIA/USI/SUPSI

摘要

METAGPT: 多智能体协作框架中的元编程摘要在基于大型语言模型 (LLMs) 的智能体社会中, 自动问题解决已取得显著进展。现有的基于LLMs的多智能体系统已能够解决简单的对话任务。然而, 对于更复杂的任务, 由于链式调用LLMs引发的幻觉导致逻辑不一致, 解决方案变得复杂。在此, 我们提出了MetaGPT, 一种创新的元编程框架, 将高效的人类工作流程融入基于LLMs的多智能体协作中。MetaGPT将标准操作程序 (SOP) 编码为提示序列, 以实现更流畅的工作流程, 从而使具有类人领域专业知识的智能体能够验证中间结果并减少错误。MetaGPT采用装配线范式, 为不同智能体分配多样化角色, 有效将复杂任务拆解为涉及多智能体协作的子任务。在协作软件工程基准测试中, MetaGPT生成的解决方案比以往的基于聊天的多智能体系统更具连贯性。我们的项目可在 <https://github.com/geekan/MetaGPT>。

1 引言

自主智能体利用大型语言模型 (LLMs) 为增强和复制人类工作流程提供了有前景的机遇。然而, 在实际应用中, 现有系统 (Park et al., 2023; Zhuge et al., 2023; Cai et al., 2023; Wang et al., 2023c; Li et al., 2023; Du et al., 2023; Liang et al., 2023; Hao et al., 2023; Zhou et al., 2023b) 往往过于简化复杂性, 难以实现有效、连贯且准确的问题解决过程, 尤其是在需要有意义的协作互动时 (Chen et al., 2024; Zhang et al., 2023a; Dong et al., 2023; Zhou et al., 2023a; Qian et al., 2023; Tang et al., 2023b; Hong et al., 2024)。

通过广泛的协作实践, 人类在各个领域开发出了广泛认可的标准操作程序 (SOPs) (Belbin, 2012; Manifesto, 2001; DeMarco & Lister, 2013)。这些SOPs在支持任务分解和高效协调方面发挥着关键作用。此外, SOPs明确了每个团队成员的职责, 同时建立了中间产出标准。明确的SOPs有助于任务的持续和准确执行, 确保其符合既定角色和质量标准 (Belbin, 2012; Manifesto, 2001; DeMarco & Lister, 2013; Wooldridge & Jennings, 1998)。例如, 在软件公司中, 产品经理分析竞争和用户需求, 使用标准化结构创建产品需求文档 (PRD), 以指导开发过程。

METAGPT: 多智能体协作框架的元编程受到此类思想的启发, 我们设计了一个基于GPT的有前景的元编程框架, 称为MetaGPT, 该框架在很大程度上受益于SOPs。与其他工作 (Li等人, 2023; Qian等人, 2023) 不同, MetaGPT要求智能体生成结构化输出, 例如高质量的需求

这些作者对本工作贡献相等。

吴成林 (alexanderwu@fuzhi.ai) 是通讯作者, 隶属于DeepWisdom。

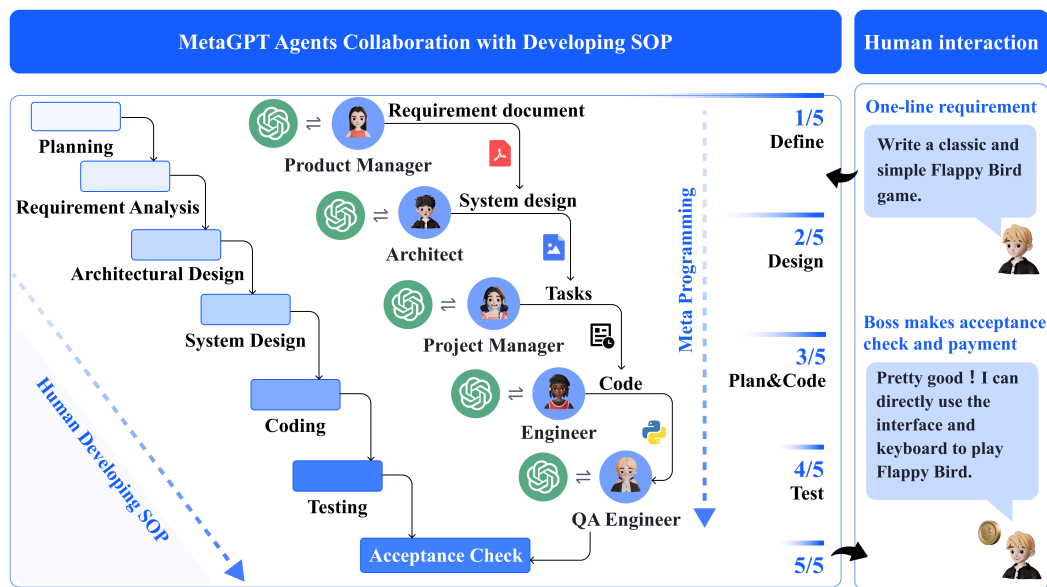


图1: MetaGPT与现实人类团队之间的软件开发标准操作程序 (SOPs)。在软件工程中, 标准操作程序促进了各角色之间的协作。MetaGPT展示了其将复杂任务分解为具体可操作程序的能力, 这些程序被分配给不同的角色 (例如, 产品经理、架构师、工程师等)。

文档、设计工件、流程图和界面规范。使用中间结构化输出显著提高目标代码生成的成功率, 因为它有助于保持沟通的一致性, 减少在协作过程中出现的歧义和错误。从更直观的角度来看, 在由MetaGPT模拟的公司中, 所有员工都遵循严格且高效的工作流程, 所有交接都必须符合一定的既定标准。这降低了由LLMs之间闲聊引起的幻觉风险, 特别是在角色扮演框架中, 例如: “你好, 你好吗?” —— Alice (产品经理); “太好了! 你吃午饭了吗?” —— Bob (架构师)。

受益于SOPs, MetaGPT为元编程提供了一种有前景的方法。在此背景下, 我们采用元编程“编程以编程”的概念, 区别于更广泛的元学习和“学习如何学习” (Schmidhuber, 1987; 1993a; Hochreiter等, 2001; Schmidhuber, 2006; Finn等, 2017)。

这种元编程的概念也包括早期的努力, 如CodeBERT (Feng等人, 2020) 以及近期的项目, 如CodeLlama (Rozière等人, 2023) 和WizardCoder (Luo等人, 2023)。然而, MetaGPT作为一种独特的解决方案, 通过一组结构合理的专业代理实现了高效的元编程。每个代理都具有特定的角色和专业知识, 遵循一些既定标准。这使得在运行时可以实现自动需求分析、系统设计、代码生成、修改、执行和调试, 突显了基于代理的技术在增强元编程方面的潜力。

为了验证MetaGPT的设计, 我们采用了公开的HumanEval (Chen等人, 2021a) 和MBPP (Austin等人, 2021) 进行评估。值得注意的是, 在代码生成基准测试中, MetaGPT在Pass@1指标上达到了85.9%和87.7%, 创下了新的最先进水平 (SoTA)。与其他用于构建复杂软件项目的流行框架相比, 如AutoGPT (Torantulino等人, 2023)、LangChain (Chase, 2022)、AgentVerse (Chen等人, 2023) 以及ChatDev (Qian等人, 2023) 等, MetaGPT在处理更高层次的软件复杂性和提供丰富功能方面也表现出色。在我们的实验评估中, MetaGPT实现了100%的任务完成率, 充分展示了其设计的稳健性和高效性 (时间和令牌成本)。

我们将我们的贡献总结如下:

<https://en.wikipedia.org/w/index.php?title=元编程>

METAGPT: 基于大模型的多智能体协作元编程框架

我们在MetaGPT设计中创新性地整合了类人标准操作程序（SOPs），显著增强了其鲁棒性，减少了基于大模型（LLM）代理之间的非生产性协作。此外，我们引入了一种新颖的执行反馈机制，在运行时调试和执行代码，显著提升了代码生成质量（例如，在MBPP任务中实现了5.4%的绝对提升）。

我们在 HumanEval (Chen 等人, 2021a) 和 MBPP (Austin 等人, 2021) 上取得了最先进的性能。大量的实验结果有力地验证了 MetaGPT，表明它是一种有前景的元编程框架，用于开发基于大模型的多智能体系统。

2 相关工作

自动编程自动编程的起源可以追溯到上个世纪。在1969年，Waldinger & Lee (1969) 提出了“PROW”系统，该系统旨在接受用谓词演算编写的程序规范，生成算法，并创建LISP实现 (McCarthy, 1978)。Balzer (1985) 和 Soloway (1986) 致力于推动自动编程的发展，并识别出实现其潜在的方法。近年来的方法则采用自然语言处理 (NLP) 技术 (Ni等, 2023; Skreta等, 2023; Feng等, 2020; Li等, 2022; Chen等, 2018; 2021b; Zhang等, 2023a; Liu等, 2023b; Tang等, 2023a; Muennighoff等, 2023)。自动编程已发展成为一个行业，提供诸如 Microsoft Copilot 等付费功能。最近，基于大规模语言模型 (LLMs) 的代理 (Yao等, 2022; Shinn等, 2023; Lin等, 2023) 推动了自动编程的发展。其中，ReAct (Yao等, 2022) 和 Reflexion (Shinn等, 2023) 利用链式思维提示 (Wei等, 2022) 生成推理轨迹和行动计划，借助 LLMs 实现。两者都展示了 ReAct 风格的推理循环作为赋能自动编程的设计范式的有效性。此外，ToolFormer (Schick等, 2023) 和 ToolLLM (Qin等, 2023) 可以通过简单的 API 学习如何使用外部工具。与我们的工作最为相关的研究由 Li 等 (2023) 提出了一种简单的角色扮演框架，用于涉及不同角色之间通信的编程。Qian 等 (2023) 则利用多个代理进行软件开发。尽管现有论文 (Li 等, 2023; Qian 等, 2023) 提高了生产效率，但尚未充分挖掘具有结构化输出格式的高效工作流程，这使得处理复杂的软件工程问题变得更加困难。

基于 LLM 的多智能体框架近年来，基于 LLM 的自主智能体在工业界和学术界都引起了极大的关注 (Wang et al., 2023b; Zhou et al., 2023b; Zhang et al., 2023b)。许多工作 (Chen et al., 2024; Wang et al., 2023c; Du et al., 2023; Zhuge et al., 2023; Hao et al., 2023; Akata et al., 2023; Tang et al., 2023b) 通过在多个智能体之间进行讨论，提升了 LLM 的问题解决能力。Stable-Alignment (Liu et al., 2023a) 通过在沙箱中与 LLM 智能体的交互，达成价值判断的共识，从而创建指令数据集。其他研究则关注社会学现象。例如，Generative Agents (Park et al., 2023) 创建了一个由 25 个智能体组成的“城镇”，以研究语言交互、社会理解和集体记忆。在基于自然语言的心智社会 (NL-SOM) (Zhuge et al., 2023) 中，不同功能的智能体通过多轮“心智风暴”互动以解决复杂任务。Cai et al. (2023) 提出了一种通过将大型模型作为工具制造者、小型模型作为工具使用者相结合的模式，以实现成本降低。

一些研究强调与规划和策略相关的合作与竞争 (Bakhtin 等人, 2022)；另一些则提出基于大模型的经济体系 (Zhuge 等人, 2023)。这些工作关注开放世界中的人类行为模拟，而 MetaGPT 旨在将人类实践引入多智能体框架。此外，基于大模型的智能体面临“助手重复指令”或“消息无限循环”的挑战 (Talebirad & Nadiri, 2023; Li 等人, 2023)。这些挑战在任务导向的协作中变得更加紧迫，因为这类协作需要一致且互利的互动 (Elazar 等人, 2021; Wang 等人, 2022; Jiang 等人, 2023)。这促使我们将诸如软件开发中的标准操作程序等先进概念应用于多智能体框架。

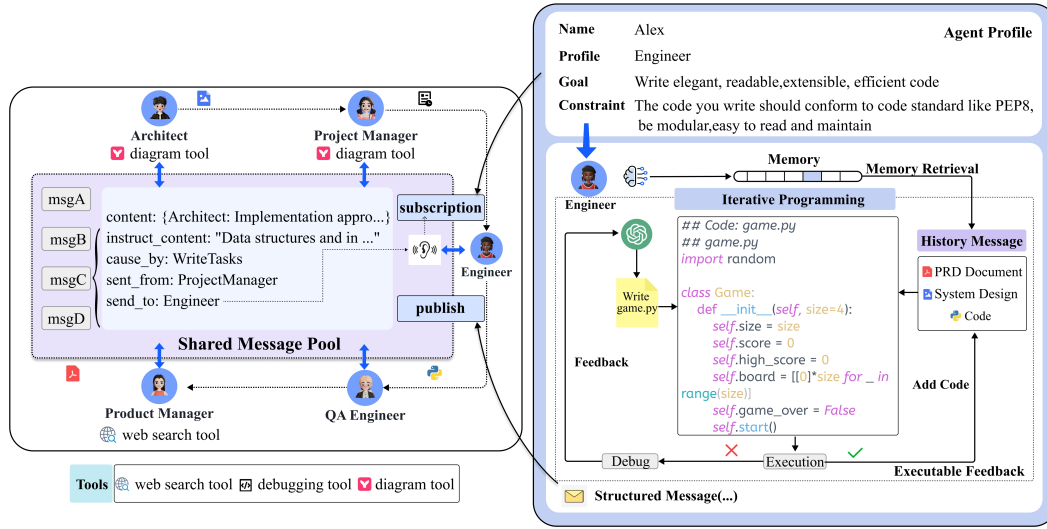


图2：通信协议示例（左）与带可执行反馈的迭代编程（右）。左：代理使用共享消息池发布结构化消息，并根据其配置文件订阅相关消息。右：在生成初始代码后，工程师代理运行并检查错误。如出现错误，代理会检查存储在内存中的历史消息，并将其与PRD、系统设计和代码文件进行比较。

3 METAGPT：一种元编程框架

MetaGPT 是一个基于大规模语言模型（LLM）的多智能体系统的元编程框架。第 3.1 节介绍了该框架中角色专业化、工作流程和结构化通信的解释，并说明了如何在标准操作程序（SOPs）的背景下组织多智能体系统。第 3.2 节提出了一种增强角色通信效率的通信协议。我们还实现了结构化通信接口和高效的发布-订阅机制。这些方法使智能体能够从其他角色获取定向信息，以及从环境中获取公共信息。最后，我们在第 3.3 节引入了可执行反馈——一种在运行时进一步提升代码生成质量的自我校正机制。

3.1 标准操作程序中的代理

角色专业化明确的角色专业化使得将复杂工作拆分为更小、更具体的任务成为可能。解决复杂任务或问题通常需要具有不同技能和专业知识的代理人协作，每个代理人贡献针对特定问题的专业输出。

在软件公司中，产品经理通常负责业务导向的分析和洞察，而软件工程师则负责编程工作。我们在软件公司中定义了五个角色：产品经理、架构师、项目经理、工程师和质量保证工程师，如图所示

In a software company, a Product Manager typically conducts business-oriented analysis and derives insights, while a software engineer is responsible for programming. We define five roles in our software company: Product Manager, Architect, Project Manager, Engineer, and QA Engineer, as shown in Figure 1. 在MetaGPT中，我们为每个角色指定代理的档案，包括其姓名、简介、目标和约束条件。我们还为每个角色初始化了具体的上下文和技能。例如，产品经理可以使用网页搜索工具，而工程师可以执行代码，如图所示2。所有代理都遵循如 Yao 等人(2022) 所描述的React风格行为。

每个代理监控环境（即，MetaGPT中的消息池）以发现重要的观察（例如，来自其他代理的消息）。这些消息可以直接触发操作，或协助完成任务。

跨代理的工作流程通过定义代理的角色和操作技能，我们可以建立基本的工作流程。在我们的工作中，遵循软件开发的标准操作程序（SOP），使所有代理能够以顺序方式协作。



图3: 展示MetaGPT软件开发流程的示意图, 强调其对标准操作程序 (SOPs) 的重要依赖。更详细的演示可参见附录B。

具体而言, 如图所示1, 在获取用户需求后, 产品经理进行全面分析, 制定包含用户故事和需求池的详细产品需求文档 (PRD), 作为初步的功能拆解。结构化的PRD随后交由架构师, 将需求转化为系统设计组件, 如文件列表、数据结构和接口定义。一旦系统设计完成, 相关信息传递给项目经理进行任务分配。工程师按照设计方案执行相应的类和函数 (详见图2)。在下一阶段, 质量保证工程师制定测试用例, 以确保代码质量。在最后一步, MetaGPT生成一份精心设计的软件解决方案。我们提供了详细的示意图 (图3) 以及MetaGPT中SOP工作流程的具体实例 (附录B)。

3.2 通信协议

结构化通信接口目前大多数基于大型语言模型 (LLM) 的多智能体框架 (Li 等人, 2023; Zhuge 等人, 2023; Zhang 等人, 2023a; Park 等人, 2023) 将自然语言作为通信接口, 且不加限制。

然而, 尽管自然语言具有多样性, 但一个问题随之而生: 纯粹的自然语言交流是否足以解决复杂任务? 例如, 在电话游戏 (或中文)

经过多轮沟通后，原始信息可能会被严重扭曲。受到人类社会结构的启发，我们提出采用结构化通信来制定代理的通信方式。我们为每个角色建立了一个模式和格式，并要求个体根据其特定角色和上下文提供必要的输出。

如图所示3，架构师代理生成两个输出：系统界面设计和流程图。这些内容包括系统模块设计和交互序列，作为工程师的重要交付物。与ChatDev (Zhao 等人, 2023) 不同，MetaGPT中的代理通过文档和图表（结构化输出）进行交流，而非对话。这些文档包含所有必要信息，避免了无关或缺失的内容。

发布-订阅机制在协作中，共享信息至关重要。例如，架构师和工程师通常需要参考PRD。然而，如前述工作所示 (Li et al., 2023; Zhao et al., 2023; Zhang et al., 2023a)，每次采用一对一的沟通方式可能会使通信拓扑变得复杂，从而导致效率低下。

为了应对这一挑战，一种可行的方法是将信息存储在全局<message pool>中。如图2（左）所示，我们引入了一个共享消息池，允许所有代理直接交换消息。这些代理不仅发布它们的结构化消息到池中，还可以透明地访问其他实体的消息。任何代理都可以直接从共享池中检索所需信息，无需询问其他代理或等待其响应，从而提高了通信效率。

与每个代理共享所有信息可能导致信息过载。在任务执行过程中，代理通常倾向于只接收与任务相关的信息，避免被无关细节分散注意力。有效的管理和传播这些信息起着关键作用。我们提供一种简单而有效的解决方案——订阅机制(在图 2 (左侧)。代理不依赖对话，而是利用角色特定的兴趣来提取相关信息。它们可以根据角色配置文件选择关注的信息。在实际实现中，代理只有在接收到所有前置依赖后，才会激活其行动。如图 3所示，架构师主要关注产品经理提供的PRD，而来自QA工程师等角色的文档可能关注度较低。

3.3 具有可执行反馈的迭代编程

在日常编程任务中，调试与优化过程扮演着重要角色。然而，现有方法往往缺乏自我纠正机制，导致代码生成不成功。此前的工作引入了不可执行的代码审查与自我反思 (Zhao et al., 2023; Yao et al., 2022; Shinn et al., 2023; Dong et al., 2023)。然而，它们在确保代码可执行性和运行时正确性方面仍面临挑战。

我们的首次MetaGPT实现未能在审查过程中发现某些错误，原因在于大型语言模型的幻觉 (Manakule et al., 2023)。为此，在初始代码生成之后，我们引入了可执行的反馈机制，以迭代改进代码。更具体地，正如图2所示，工程师被要求根据原始产品需求和设计编写代码。

这使得工程师能够利用自身的历史执行和调试记忆不断优化代码。为了获取更多信息，工程师编写并执行相应的单元测试用例，随后获得测试结果。如果结果令人满意，则启动额外的开发任务；否则，工程师在继续编程之前会对代码进行调试。这个迭代测试过程持续进行，直到测试通过或最多重试3次为止。

4 实验

4.1 实验设置

数据集我们使用两个公共基准测试，HumanEval (Chen 等人, 2021a) 和 MBPP (Austin 等人, 2021)，以及一个自生成的、更具挑战性的软件开发基准测试，名为 Soft-

https://en.wikipedia.org/wiki/Chinese_whispers

METAGPT: 多智能体协作框架的元编程wareDev: (1) HumanEval 包含164个手写编程任务。这些任务涵盖函数规范、描述、参考代码和测试。(2) MBPP由427个Python任务组成。这些任务涉及核心概念和标准库特性, 包括描述、参考代码和自动化测试。(3) 我们的SoftwareDev数据集是70个具有代表性的软件开发任务示例的集合, 每个任务都配有任务提示(见表8)。这些任务范围多样(见图5), 如迷你游戏、图像处理算法、数据可视化。它们为真实的开发任务提供了坚实的测试平台。与之前的数据集(Chen等人, 2021a; Austin等人, 2021)相反, SoftwareDev侧重于工程方面。在比较中, 我们随机选择七个具有代表性的任务进行评估。

评估指标对于HuamnEval和MBPP, 我们遵循(Pass@ k)由(Chen等人, 2021a; Dong等人, 2023)提出的无偏版本, 用于评估前 k 个生成代码的功能准确率: Pass@

$$k = \mathbb{E}_{\text{Problems}} \left[1 - \binom{n-c}{k} \binom{n}{k} \right].$$

对于SoftwareDev, 我们优先考虑实际应用, 并通过人工评估(A、E)或统计分析(B、C、D)来评估性能: **(A)**可执行性: 该指标对代码进行评分, 范围从1(失败/不可用)到4(完美无瑕)。“1”表示不可用, “2”表示可运行但不完美, “3”接近完美, “4”则为无瑕疵代码。**(B)**成本: 此处的成本评估包括(1)运行时间, (2)令牌使用量, 以及(3)开销。**(C)**代码统计: 包括(1)代码文件数, (2)每个文件的代码行数, 以及(3)总代码行数。**(D)**生产率: 基本定义为令牌使用量除以代码行数, 即每行代码的令牌消耗。**(E)**人工修正成本: 指手动代码修正的次数, 解决的问题包括包导入错误、类名错误或引用路径不完整。通常每次修正涉及最多3行代码。

基线方法 我们将我们的方法与近年来在代码生成领域的特定领域大型语言模型(LLMs)进行比较, 包括AlphaCode(Li等, 2022)、InCoder(Fried等, 2022)、CodeGeeX(Zheng等, 2023)、CodeGen(Nijkamp等, 2023)、CodeX(Chen等, 2021a)以及CodeT(Chen等, 2022), 以及通用领域的LLMs, 如PaLM(Chowdhery等, 2022)和GPT-4(OpenAI, 2023)。一些基线(如InCoder、CodeGeeX)的结果由Dong等(2023)提供。在HumanEval和MBPP中, 我们对提示进行了轻微修改, 以符合响应格式要求。这些修改旨在解决特定格式的问题(即Python问题)。在SoftwareDev基准测试中, 我们对MetaGPT、AutoGPT(Torantulino等, 2023)、LangChain(Chase, 2022)结合Python Read-Eval-Print Loop(REPL)工具³, Agent-Verse(Chen等, 2023)以及ChatDev(Qian等, 2023)进行了全面比较。

4.2 主要结果

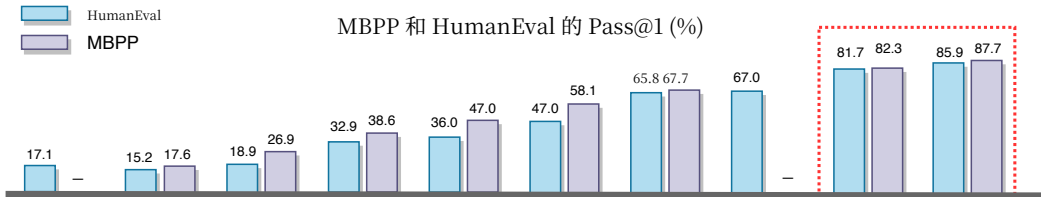


图4: 在单次尝试下, MBPP和HumanEval的通过率。

性能 图 4 表明MetaGPT在HumanEval和MBPP基准测试中均优于所有前沿方法。当MetaGPT与GPT-4合作时, 在HumanEval基准测试中的Pass @ k 指标明显优于GPT-4, 分别达到85.9%和87.7%

https://en.wikipedia.org/wiki/Read-eval-print_loop

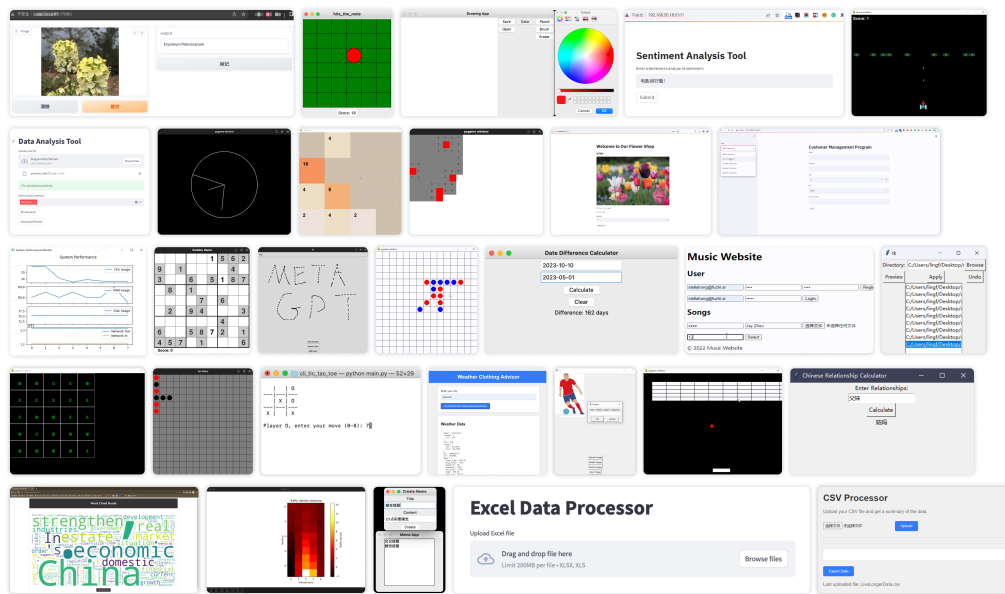


图5: MetaGPT开发的演示软件。

在这两个公共基准测试中。此外，如表所示，MetaGPT在几乎所有指标上都优于ChatDev，特别是在具有挑战性的SoftwareDev数据集上。例如，在可执行性方面，MetaGPT的得分为3.75，接近满分4（完美无瑕）。此外，它所需时间（503秒）明显少于ChatDev。考虑到代码统计和人工修订的成本，MetaGPT也显著优于ChatDev。虽然MetaGPT所需的tokens更多（24,613或31,255对比19,292），但生成一行代码只需126.5/124.3个tokens，而ChatDev则使用248.9个tokens。这些结果凸显了SOP在多智能体协作中的优势。此外，我们通过可视化样本（图5）展示了MetaGPT的自主软件生成能力。如需更多实验和分析，请参阅附录C。

表1: SoftwareDev的统计分析

统计指标	ChatDev	MetaGPT 无反馈	MetaGPT
(A) 可执行性	2.25	3.67	3.75
(B) 成本#1: 运行时间（秒）	762	503	541
(B) 成本#2: 令牌使用量	19,292	24,613	MetaGPT: 多智能体协作框架的元编程
(C) 代码统计#1: 代码文件	1.9	4.6	5.1
(C) 代码统计#2: 每个文件的代码行数	40.8	42.3	49.3
(C) 代码统计#3: 总代码行数	77.5	194.6	251.4
生产力	248.9	126.5	124.3
人类修订成本	2.5	2.25	0.83

4.3 能力分析

与开源基线方法如AutoGPT以及自主代理如AgentVerse和ChatDev相比，MetaGPT提供了用于软件工程任务的功能。正如表2所示，我们的框架具备广泛的能力，能够高效处理复杂和专业化的开发任务。引入标准操作程序（例如角色扮演专业知识、结构化沟通、流水线型工作流程）可以显著提升代码生成效果。其他基线方法

<Table 2>: MetaGPT与其他方法能力的比较 ‘✓’ 表示 the 在相应框架中存在特定特征, ‘✗’ 其缺失。

框架能力	AutoGPT 1				MetaGPT
PRD 生成	✗	✗	✗	✗	✓
技术设计生成	✗	✗	✗	✗	✓
API接口生成	✗	✗	✗	✗	✓
代码生成	✓	✓	✓	✓	✓
预编译执行	✗	✗	✗	✗	✓
基于角色的任务管理	✗	✗	✗	✓	✓
代码审查	✗	✗	✓	✓	✓

<Table 3: ></Table 3: >角色消融研究。‘#’ 表示 ‘数量’, ‘Product’ 表示 ‘产品经理’, ‘Project’ 表示 ‘项目经理’。’ 表示添加了特定角色。‘Revisions’ 指的是 ‘HumanRevisionCost’。

工程师 产								
✓	✗	✗	✗	1	83.0	\$ 0.915	10	1.0
✓	✓	✗	✗	2	112.0	\$ 1.059	6.5	2.0
✓	✓	✓	✗	3	143.0	\$ 1.204	4.0	2.5
✓	✓	✗	✓	3	205.0	\$ 1.251	3.5	2.0
✓	✓	✓	✓	4	191.0	\$ 1.385	2.5	4.0

can 轻松集成类似SOP的设计以提升性能, 类似于注入链式结构 thought (Wei et al., 2022) in LLMs.

4.4 消融研究

角色的有效性为了理解不同角色对最终结果的影响, 我们执行了两个任务, 涉及生成有效代码和计算平均统计数据。当我们排除某些角色时, 会生成不可行的代码。如表所示, 除了工程师之外的角色的加入始终能改善修订次数和可执行性。虽然增加更多角色会略微提高成本, 但整体性能明显提升, 证明了各种角色的有效性。

可执行反馈机制的有效性如图所示4, 将可执行反馈引入MetaGPT在Pass@1指标上分别在Hu- manEval和MBPP上实现了4.2%和5.4%的显著提升。此外, 表格1 显示, 反馈机制提高了可行性 (3.67提升至3.75) 并降低了人工修订的成本 (2.25降至0.83)。这些结果说明我们设计的反馈机制能够生成更高质量的代码。MetaGPT及无可执行反馈的MetaGPT的额外定量结果见表格4 和表格9。

5 结论

METAGPT: 多智能体协作框架的元编程

致谢

我们感谢KAUST人工智能倡议的执行秘书Sarah Salhi和KAUST人工智能倡议的博士后研究员Yuhui Wang在润色部分文本方面的帮助。我们还要感谢KAUST人工智能倡议的博士生Wenyi Wang, 提供了全面的反馈意见, 并协助与Mingchen共同起草了展望(附录A)。同时, 感谢DeepWisdom副总裁Zongze Xu提供的AgentStore示意材料。

作者贡献

宏思睿完成了大部分实验并设计了可执行反馈模块。在Ceyao Zhang的支持下, 她还领导了初稿的撰写, 同时得到Jinlin Wang和Zili Wang的协助。朱格明辰设计了自我提升模块, 讨论了额外的实验, 并主导了当前的论文撰写。陈佳琪协助完成了MBPP实验, 概述了方法部分, 并为当前稿件做出了贡献。郑夏武提供了宝贵的指导, 审阅并编辑了论文。程宇恒参与了评估指标的设计和HumanEval实验。游嘉成、林子娟、周立阳、肖凌峰协助完成了MBPP实验及与开源基线方法的对比。冉晨宇制作了大部分示意图。吴成林是DeepWisdom的首席执行官, 发起了MetaGPT项目, 为其贡献了大量代码, 并为本项目提供了建议。Jurgen Schmidhuber, KAUST人工智能计划主管兼IDSIA科学总监, 提供了指导并协助论文撰写。

参考文献

- Elif Akata, Lion Schulz, Julian Coda-Forno, Seong Joon Oh, Matthias Bethge, 和 Eric Schulz. 使用大型语言模型进行重复博弈。 *arXiv预印本*, 2023年。
- Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, and Charles Sutton. 程序合成与大型语言模型, 2021。
- Anton Bakhtin, Noam Brown, Emily Dinan, Gabriele Farina, Colin Flaherty, Daniel Fried, Andrew Goff, Jonathan Gray, Hengyuan Hu 等。通过结合语言模型与策略推理实现人类水平的外交游戏表现。 *Science*, 2022年。
- Robert Balzer. 自动编程的十五年展望。 *TSE*, 1985年。
- R.M. Belbin. 《团队角色在工作中的应用》. Routledge, 2012. URL <https://books.google.co.uk/books?id=MHIQBAAQBAJ>.
- Tianle Cai, Xuezhi Wang, Tengyu Ma, Xinyun Chen 和 Denny Zhou. 大型语言模型 as 工具制造商。 *arXiv预印本*, 2023年。
- Harrison Chase. LangChain. <https://github.com/hwchase17/langchain>, 2022。
- Bei Chen, Fengji Zhang, Anh Nguyen, Daoguang Zan, Zeqi Lin, Jian-Guang Lou, 和 Weizhu Chen. Codet: 结合生成测试的代码生成, 2022年。
- Jiaqi Chen, Yuxian Jiang, Jiachen Lu, and Li Zhang. S-agents: 开放式自组织代理 环境。 *arXiv预印本*, 2024年。
- 马克·陈、杰瑞·特沃克、许武俊、袁启明、Henrique Ponde de Oliveira Pinto、贾里德·卡普兰、哈里·爱德华兹、尤里·布尔达、尼古拉斯·约瑟夫、格雷格·布罗克曼、亚历克斯·雷、劳尔·普里、格雷琴·克鲁格、迈克尔·佩特罗夫、海迪·克拉夫、吉里什·萨斯特里、帕梅拉·米什金、布鲁克·陈、斯科特·格雷、尼克·赖德、米哈伊尔·帕夫洛夫、阿莱西亚·鲍尔、卢卡斯·凯撒、穆罕默德·巴维亚然、克莱门斯·温特、菲利普·蒂莱、费利佩·佩特罗斯基·苏奇、戴夫·卡明斯、马蒂亚斯·普拉普特、福蒂奥斯·昌齐斯、伊丽莎白·巴恩斯、阿里尔·赫伯特·沃斯、威廉·赫布根·古斯、亚历克斯·尼科尔、亚历克斯·佩诺、尼克拉斯·特扎克、汤杰、伊戈尔·巴布什金、苏奇尔·巴拉吉、尚塔努·贾恩、威廉·桑德斯、克里斯托弗·赫塞、安德鲁·N·卡尔、简·莱克、乔什·阿奇姆、维丹特·米斯拉、埃文·莫里卡瓦、亚历克斯·拉德福德、马修·奈特、迈尔斯·布伦迪奇、米拉·穆拉蒂、凯蒂·迈耶、彼得·韦林德、鲍勃·麦格鲁、达里奥·阿莫代伊、山姆·麦坎德利什、伊利亚·苏茨克夫、沃伊切赫·扎莱姆巴。评估基于代码训练的大型语言模型, 2021a。

陈维泽、苏宇晟、左靖伟、杨成、袁辰飞、钱晨、陈志民、秦玉佳、卢雅熙、谢若冰、刘志远、孙茂松、周杰。Agentverse: 促进多智能体协作与探索智能体中的涌现行为, 2023。

陈新云、刘畅、宋道恩。基于执行引导的神经程序合成。发表于*ICLR*, 2018年。

陈新云, 宋晓东, 田远东。超越领域特定语言的神经程序合成的潜在执行。*NeurIPS*, 2021b。

Aakanksha Chowdhery, Sharan Narang, Jacob Devlin, Maarten Bosma, Gaurav Mishra, Adam Roberts, Paul Barham, Hyung Won Chung, Charles Sutton, Sebastian Gehrmann, Parker Schuh, Kensen Shi, Sasha Tsvyashchenko, Joshua Maynez, Abhishek Rao, Parker Barnes, Yi Tay, Noam Shazeer, Vinodkumar Prabhakaran, Emily Reif, Nan Du, Ben Hutchinson, Reiner Pope, James Bradbury, Jacob Austin, Michael Isard, Guy Gur-Ari, Pengcheng Yin, Toju Duke, Anselm Lev-skaya, Sanjay Ghemawat, Sunipa Dev, Henryk Michalewski, Xavier Garcia, Vedant Misra, Kevin Robinson, Liam Fedus, Denny Zhou, Daphne Ippolito, David Luan, Hyeontaek Lim, Barret Zoph, Alexander Spiridonov, Ryan Sepassi, David Dohan, Shivani Agrawal, Mark Omernick, Andrew M. Dai, Thanumalayan Sankaranarayana Pillai, Marie Pellat, Aitor Lewkowycz, Erica Moreira, Rewon Child, Oleksandr Polozov, Katherine Lee, Zongwei Zhou, Xuezhi Wang, Brennan Saeta, Mark Diaz, Orhan Firat, Michele Catasta, Jason Wei, Kathy Meier-Hellstern, Douglas Eck, Jeff Dean, Slav Petrov, Noah Fiedel. Palm: 利用 pathways 扩展语言模型, 2022。

T. DeMarco 和 T.R. Lister. Peopleware: 生产性项目与团队 . Addison-Wesley, 2013. URL <https://books.google.co.uk/books?id=DVIsAQAAQBAJ> . Yihong Dong, Xue Jiang, Zhi Jin, 和 Ge Li. 通过 ChatGPT 实现自我协作代码生成。 *arXiv 预印本* , 2023. Yilun Du, Shuang Li, Antonio Torralba, Joshua B. Tenenbaum, 和 Igor Mordatch. 通过多智能体辩论提升语言模型的事实性与推理能力, 2023。

Yanai Elazar, Nora Kassner, Shauli Ravfogel, Abhilasha Ravichander, Eduard Hovy, Hinrich Schütze, 和 Yoav Goldberg. 测量与提升预训练语言模型中的一致性。 *TACL* , 2021年。

Zhangyin Feng, Daya Guo, Duyu Tang, Nan Duan, Xiaocheng Feng, Ming Gong, Linjun Shou, Bing Qin, Ting Liu, Daxin Jiang, 等. Codebert: 一种用于编程语言和自然语言的预训练模型。 *arXivpreprint* , 2020。

Chrisantha Fernando, Dylan Banarse, Henryk Michalewski, Simon Osindero, and Tim Rocktäschel. Promptbreeder: 通过提示演化实现自我引用的自我改进。 *arXiv预印本* , 2023年。Chelsea Finn, Pieter Abbeel, 和 Sergey Levine。面向快速适应的模型无关元学习。发表于国际机器学习会议 (*ICML*) , 2017年。Daniel Fried, Armen Aghajanyan, Jessy Lin, Sida Wang, Eric Wallace, Freda Shi, Ruiqi Zhong, Wen-tau Yih, Luke Zettlemoyer, 和 Mike Lewis. InCoder: 一种用于代码填充和合成的生成模型。 *arXiv预印本* , 2022年。Irving John Good. 关于第一台超智能机器的猜测。 *计算机科学进展* , 1965年。Rui Hao, Linmei Hu, Weijian Qi, Qingliu Wu, Yirui Zhang, 和 Liqiang Nie. Chatllm网络: 更多大脑, 更多智能。 *arXiv预印本* , 2023年。S. Hochreiter, A. S. Younger, 和 P. R. Conwell. 利用梯度下降进行学习的学习方法。发表于计算机科学讲义第2130卷, 国际人工神经网络会议 (*ICANN-2001*) 论文集, 第 87-94页。Springer: 柏林, 海德堡, 2001年。Sirui Hong, Yizhang Lin, Bangbang Liu, Binhao Wu, Danyang Li, Jiaqi Chen, Jiayi Zhang, Jinlin Wang, Lingyao Zhang, Mingchen Zhuge 等。数据解释器: 一种用于数据科学的LLM代理。 *arXiv预印本 arXiv:2402.18679* , 2024年。

薛江、董一宏、王乐成、商启伟、李格。基于大规模语言模型的自主规划代码生成。*arXiv*预印本，2023年。

李国豪、哈桑·阿贝德·阿尔·卡德尔·哈穆德、哈尼·伊塔尼、德米特里·赫兹布林、伯纳德·加南。Camel: 用于大型语言模型社会“心智”探索的交流代理。*arXiv*预印本，2023年。

Yujia Li, David Choi, Junyoung Chung, Nate Kushman, Julian Schrittwieser, Rémi Leblond, Tom Eccles, James Keeling, Felix Gimeno, Agustin Dal Lago 等人。基于AlphaCode的竞赛级代码生成。*Science*, 2022年。

田亮、何志伟、焦文祥、王兴、王燕、王睿、杨玉玖、涂兆鹏、史树明。通过多智能体辩论促进大型语言模型的发散性思维。*arXiv*预印本，2023年。

Bill Yuchen Lin, Yicheng Fu, Karina Yang, Prithviraj Ammanabrolu, Faeze Brahman, Shiyu Huang, Chandra Bhagavatula, Yejin Choi, 和 Xiang Ren. Swiftsage: 一种具有快速与缓慢思维的生成代理，用于复杂交互任务。*arXivpreprint*, 2023年

刘瑞博，杨瑞欣，贾晨燕，张歌，周丹尼，戴安德鲁·M，杨迪伊，和索鲁什·沃苏吉。在模拟人类社会中训练社会对齐的语言模型。*arXivpreprint*, 2023a。

刘玉良，唐翔如，蔡泽凡，卢俊杰，张一驰，邵彦军，邓泽轩，胡荷兰，杨增贤，安凯凯 等。Ml-bench: 大型语言模型利用开源库进行机器学习任务。*arXivpreprint arXiv:2311.09835*, 2023b。

Ziyang Luo, Can Xu, Pu Zhao, Qingfeng Sun, Xiubo Geng, Wenxiang Hu, Chongyang Tao, Jing Ma, Qingwei Lin, 和 Daxin Jiang. Wizardcoder: 通过 evol-instruct 提升代码大模型的能力。*arXivpreprint*, 2023。

Potsawee Manakul, Adian Liusie, 和 Mark JF Gales. Selfcheckgpt: 面向生成式大规模语言模型的零资源黑箱幻觉检测。*arXiv*预印本，2023年。

敏捷宣言。敏捷软件开发宣言。犹他州斯诺伯德，2001年。

约翰·麦卡锡。Lisp的历史。收录于编程语言的历史。1978年。

Niklas Muennighoff, Qian Liu, Armel Zebaze, Qinkai Zheng, Binyuan Hui, Terry Yue Zhuo, Swayam Singh, Xiangru Tang, Leandro Von Werra, 和 Shayne Longpre. Octopack: 指令调优大型语言模型。*arXivpreprint arXiv:2308.07124*, 2023年。

Ansong Ni, Srini Iyer, Dragomir Radev, Veselin Stoyanov, Wen-tau Yih, Sida Wang, 和 Xi Victoria Lin. Lever: 学习通过执行验证语言到代码生成。在*ICML*, 2023年。

Erik Nijkamp, Bo Pang, Hiroaki Hayashi, Lifu Tu, Huan Wang, Yingbo Zhou, Silvio Savarese, 和 Caiming Xiong. Codegen: 一种用于代码的开源大规模语言模型，支持多轮程序合成，2023年。

OpenAI. GPT-4技术报告，2023年。

Joon Sung Park, Joseph C O’ Brien, Carrie J Cai, Meredith Ringel Morris, Percy Liang, 和 Michael S Bernstein. 生成代理：人类行为的交互模拟体。*arXivpreprint*, 2023年。

陈谦，辛聪，程阳，魏泽辰，苏玉生，许巨源，刘志远，孙茂松。用于软件开发的交互式智能体，2023。

Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian 等。Toollm: 促进大型语言模型掌握 16000+ 实际应用接口。*arXiv* 预印本 *arXiv:2307.16789*, 2023年。

Baptiste Rozière, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, Xiaoqing Ellen Tan, Yossi Adi, Jingyu Liu, Tal Remez, J’eremy Rapin 等。Code llama: 开源基础模型用于代码。*arXivpreprint*, 2023年。

Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda 以及 Thomas Scialom. Toolformer: 语言模型可以自我学习使用工具。arXiv预印本, 2023年。J. Schmidhuber. 自指权重矩阵。发表于《国际人工神经网络会议论文集》, 阿姆斯特丹, 第446–451页。Springer, 1993a。J. Schmidhuber. Gödel机器: 自指的通用问题解决者, 能实现可证明的最优自我改进。技术报告IDSIA-19-03, arXiv:cs.LO/0309048 v3, IDSIA, Manno-Lugano, 瑞士, 2003年12月。J. Schmidhuber. Gödel机器: 完全自指的最优通用自我改进者。在B. Goertzel 和 C. Pennachin (编), 《人工通用智能》, 第199–226页。Springer出版社, 2006年。变体可在arXiv:cs.LO/0309048获取。J. Schmidhuber. 终极认知 ‘a la Gödel. 认知计算, 1(2):177–193, 2009年。Jürgen Schmidhuber. 自指学习中的进化原则, 或关于学习如何学习: 元-元-...钩子。博士论文, 1987年。Jürgen Schmidhuber. “自指”权重矩阵。在ICANN’93: 国际人工神经网络会议论文集, 荷兰阿姆斯特丹, 1993年9月13–16日, 页3, 1993b。Jürgen Schmidhuber. 关于学习思考: 用于强化学习控制器和循环神经世界模型新组合的算法信息理论。arXiv预印本, 2015年。Jürgen Schmidhuber, Jieyu Zhao, 和 Nicol N Schraudolph. 具有自我修改策略的强化学习。发表于《学习如何学习》, 1998年。Noah Shinn, Beck Labash, 和 Ashwin Gopinath. Reflexion: 具有动态记忆和自我反思的自主代理。arXiv预印本, 2023年。Marta Skreta, Naruki Yoshikawa, Sebastian Arellano-Rubach, Zhi Ji, Lasse Bjørn Kristensen, Kourosh Darvish, Alán Aspuru-Guzik, Florian Shkurti, 和 Animesh Garg. 错误是有用的提示: 带验证器辅助的迭代提示指导任务编程。arXiv预印本, 2023年。Elliot Soloway. 学习编程 = : 构建机制和解释的学习。计算机协会通讯, 1986年。Yashar Talebirad 和 Amirhossein Nadiri. 多智能体协作: 利用智能大模型代理的力量, 2023年。Xiangru Tang, Bill Qian, Rick Gao, Jiakang Chen, Xinyun Chen, 和 Mark Gerstein. Biocoder: 具有上下文实用知识的生物信息学代码生成基准。arXiv预印本 arXiv:2308.16458, 2023a。Xiangru Tang, Anni Zou, Zhuosheng Zhang, Yilun Zhao, Xingyao Zhang, Arman Cohan, 和 Mark Gerstein. Medagents: 作为零样本医疗推理合作伙伴的大型语言模型。arXiv预印本 arXiv:2311.10537, 2023b。Torantulino 等. Auto-gpt. <https://github.com/Significant-Gravitas/Auto-GPT>, 2023年。R. J. Waldinger 和 R. C. T. Lee. PROW: 迈向自动程序编写的一步。在D. E. Walker 和 L. M. Norton (编), 《第一届国际联合人工智能会议论文集》(IJCAI), 1969年。Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, 和 Anima Anandkumar. Voyager: 具有大型语言模型的开放式体现代理。arXiv预印本, 2023a。

王雷, 马辰, 冯雪阳, 张泽宇, 杨浩, 张敬森, 陈志远, 唐济凯, 陈旭, 林彦凯等。基于大型语言模型的自主智能体综述。 *arXiv*预印本, 2023b。

王学智、魏杰森、Dale Schuurmans、Quoc Le、Ed Chi、Sharan Narang、Aakanksha Chowdhery 以及 Denny Zhou。自治性提升语言模型中的链式思维推理能力。 *arXivpreprint*, 2022年。

王振海龙, 毛少光, 吴文山, 葛涛, 韦福如, 季恒。释放大型语言模型中的认知协同: 一种通过多角色自我合作的任务解决代理。 *arXiv*预印本, 2023c。

Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou 等。链式思维提示引发大型语言模型的推理能力。 *NeurIPS*, 2022。

Michael Wooldridge 和 Nicholas R. Jennings. 面向智能体开发的陷阱. 收录于第二届自主智能体国际会议论文集, 1998. 网址 <https://doi.org/10.1145/280765.280867>.

姚顺宇, 赵杰弗里, 余殿, 杜楠, 伊扎克·沙弗兰, 卡尔蒂克·纳拉西姆汉, 曹远。React: 在语言模型中协同推理与行动。 *arXivpreprint*, 2022年。

Eric Zelikman、Eliana Lorch、Lester Mackey 和 Adam Tauman Kalai. 自我学习优化器 (stop): 递归自我改进的代码生成。 *arXiv* 预印本, 2023年

张宏新、杜伟华、单佳明、周勤宏、杜一伦、Joshua B Tenenbaum、舒天敏、甘创。基于大型语言模型的合作体现代理的模块化构建。 *arXivpreprint*, 2023a。

张卓生、姚瑶、张 Aston、唐翔如、马新贝、何志伟、王一鸣、Mark Gerstein、王睿、刘公申等。点燃语言智能: 从链式思维推理到语言代理的搭便车指南。 *arXiv preprint arXiv:2311.11797*, 2023b。

赵旭峰, 李梦迪, 康奈利乌斯·韦伯, 穆罕默德·布尔汉·哈菲兹, 和斯特凡·韦尔特。与环境对话: 基于大规模语言模型的交互式多模态感知。 *arXivpreprint*, 2023。

郑勤凯, 夏晓, 邹旭, 董宇晓, 王珊, 薛宇飞, 王子涵, 沈雷, 王安迪, 李阳, 苏腾, 杨志林, 唐杰。Codegeex: 一种用于代码生成的预训练模型, 并在 humaneval-x 上进行多语言评估, 2023年。

周书燕, 徐方福, 朱浩, 周旭辉, 罗罗, Abishek Sridhar, 程贤毅, Yonatan Bisk, Daniel Fried, Uri Alon 等。Webarena: 用于构建自主代理的逼真网页环境。 *arXiv preprint*, 2023a。

Zhou Wangchunshu, Jiang Yuchen Eleanor, Li Long, Wu Jialong, Wang Tiannan, Qiu Shi, Zhang Jintian, Chen Jing, Wu Ruipu, Wang Shuai, 等。Agents: 一个用于自主语言代理的开源框架。 *arXiv* 预印本 *arXiv:2309.07870*, 2023b。

朱格明辰, 刘浩哲, 法乔乔弗兰斯科, 迪兰·R·阿什利, R'obert Csord'as, 阿南德·戈帕拉克里希南, 阿卜杜拉·哈姆迪, 哈桑·阿贝德·卡德尔·哈穆德, 文森特·赫尔曼, 入井一树等。基于自然语言的心智社会中的心智风暴。 *arXivpreprint*, 2023。

展望

A.1 自我提升机制

本文正文中MetaGPT版本的一个局限性在于每个软件项目都是独立执行的。然而，通过积极的团队合作，软件开发团队应当从每个项目开发中积累的经验中学习，从而随着时间的推移变得更加兼容和成功。

这与递归自我改进的思想有关，最早在1965年（Good, 1965）非正式提出，首次具体实现可追溯到1987年（Schmidhuber, 1987; 1993b; Schmidhuber et al., 1998），最终形成了数学上最优的自指自我改进者的概念（Schmidhuber, 2003; 2009）。一般而言，一个系统应从现实世界的经验中学习，并通过学习经验改进学习算法的元学习，从元学习的经验中改进元学习算法，依此类推，除了可计算性和物理限制之外，没有其他限制。

近年来，一些相关工作利用大型语言模型（LLMs）的推理能力，递归优化LLMs的提示，以提升在某些下游任务中的表现（Fernando等人, 2023; Zelikman等人, 2023），类似于2015年的自适应提示工程师（Schmidhuber, 2015），其中一个神经网络学习生成一系列查询或提示，供另一个预训练神经网络使用，其回答可能帮助第一个网络更快地学习新任务。

在我们目前的工作中，我们还探索了一种自我引用机制，该机制基于代理在软件开发过程中观察到的信息，递归地修改其约束提示。我们的初步实现如下：在每个项目开始之前，软件公司的每个代理都会回顾之前的反馈，并对其约束提示进行必要的调整。这使得他们能够不断从过去的项目经验中学习，并通过改善每个个体来提升整个多智能体系统。我们首先为每个代理建立一个*handover feedback*动作。该动作负责对在之前项目开发过程中接收的信息进行批判性总结，并将这些信息整合到更新的约束提示中。总结的信息被存储在*long-term memory*中，以便在未来的约束提示更新中继承。当启动新项目时，每个代理以*react*动作开始。每个代理评估接收到的反馈，并总结出他们可以在约束提示中改进的方面。

目前的一个限制是，这些基于总结的优化仅修改角色专业化中的约束（第3.1节），而非通信协议中的结构化通信接口（第3.2节）。未来的进展尚待探索。

A.2 多智能体经济

在实际的团队合作中，交互过程通常不是硬编码的。例如，在软件公司中，协作标准操作程序（SOP）可能会动态变化。

一种自组织的实现方式在论文《基于自然语言的心智社会》（NLSOM）中进行了讨论（Zhuge et al., 2023），该论文引入了“心智经济”（EOM）的概念，这是一种面向大规模语言模型（LLMs）及其他代理的强化学习（RL）框架。EOM并非采用传统的强化学习技术通过调整神经网络参数来优化系统的总奖励，而是借鉴自由市场中的供需原则，将信用（资金）分配给那些对经济成功（奖励）做出贡献的代理。

METAGPT：多智能体协作框架的元编程DeepWisdom最近基于智能体的平台（AgentStore⁴）与EOMs的信用分配概念兼容。AgentStore中的每个智能体提供一份包含相应成本的服务列表。平台提供了便捷的API，方便人类用户或平台中的智能体轻松购买其他智能体的服务以完成任务。图6展示了AgentStore的用户界面（UI），其中展示了具有不同技能的各种智能体。此外，个人开发者可以参与构建新智能体，并在社区内实现协作开发。具体而言，AgentStore允许用户根据需求订阅智能体

<http://beta.deepwisdom.ai>

并根据其使用情况付费。此外，用户还可以购买额外的功能以扩展其现有代理的即插即用功能。这使得用户能够逐步升级他们的代理。在MetaGPT框架内，AgentStore能够支持各种代理的协作。用户可以将多个代理集成在一起，以执行更复杂的任务或项目，所有代理都共享并遵循MetaGPT中定义的开发和通信协议。

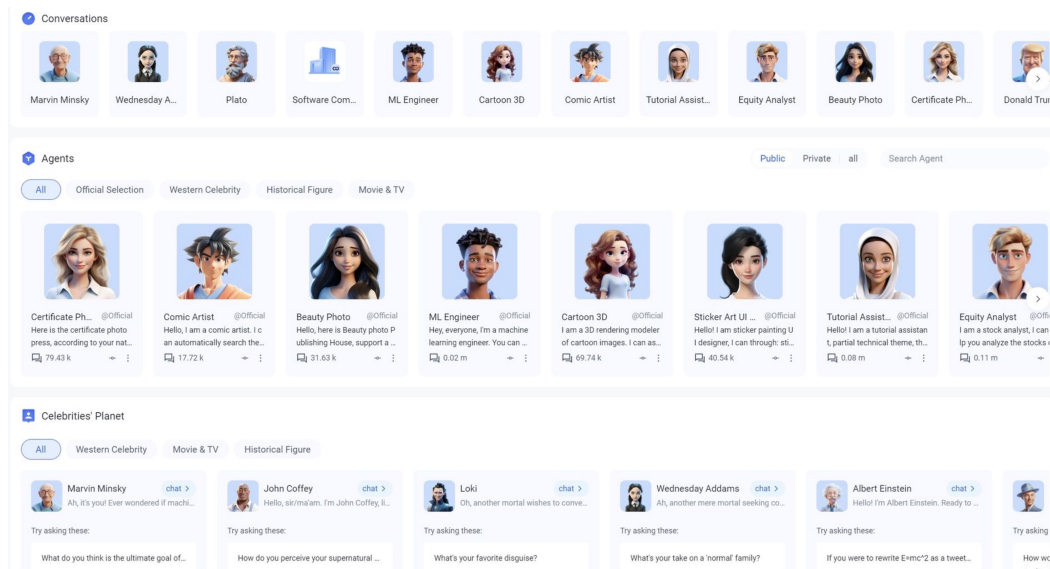


图6：AgentStore是一个专为在MetaGPT框架内创建和开发智能体而设计的平台。该平台为用户提供操作界面，便于用户轻松管理具有不同情感、个性和能力的多种智能体，以完成特定任务。

B 执行演示示例

在本节中，我们概述了使用MetaGPT进行软件开发的完整流程。该流程始于用户的输入命令（如附录 B.1 所示），以根据用户的规格设计软件为结束。

B.1 用户输入

Upo在接收用户指令时，MetaGPT 与专业开发人员合作
teaMETAGPT: 多智能体协作框架的元编程B.1 用户输入

METAGPT: 多智能体协作框架的元编程 产品需求文档

1 编写一个Python3图形界面应用程序，使用户可以用它绘制图像

B.2 METAGPT 开发流程

现在我们提供每个代理的标准化输出流程的逐步说明。

产品经理产品经理根据指定文档生成产品需求文档（PRD）。该文档包括目标、用户故事、竞争分析、需求分析和需求池。此外，还会制作竞争四象限图（见图7）。随后，这些文档和图表将交由架构师进行系统设计。

产品需求文档

1 ## 原始需求 2 主管需要一个Python3的GUI颜色计量器，能够在移动时输出RGB值。 3 4 ## 产品目标 5 ‘ ‘ ‘python7 [“创建一个用户友好的GUI颜色计量器”， 8 “确保颜色计量器能够准确输出RGB值”， 9 “确保颜色计量器在移动时实时更新RGB值” 10] 11 ‘ ‘ ‘ 12 13 ## 用户故事 14 ‘ ‘ ‘python15 [16 “作为用户，我希望能够在屏幕上选择任何颜色，以便获取其RGB值”， 17 “作为用户，我希望在移动颜色计量器时RGB值能够实时更新，以便我↔ 可以获得准确的颜色信息”， 18 “作为用户，我希望界面简单直观，以便我可以轻松使用颜色计量器” 19 20 ‘ ‘ ‘ 21 22 ## 竞争分析 23 ‘ ‘ ‘python25 [“Color Cop: 一款流行的Windows颜色拾取器。它简单易用，但缺乏实时RGB值更新”， 26 “Just Color Picker: 提供实时颜色拾取和多种颜色代码格式。然而，↔ 其界面杂乱，不太用户友好”， 27 “Instant Eyedropper: 一个驻留在系统托盘的简单颜色拾取器。它方便但↔ 缺乏高级功能”， 28 “ColorZilla: 浏览器的颜色拾取扩展。它在网页设计中很方便，但不适合↔ 桌面应用程序”， 29 “ColorPic: 提供颜色调色板和混色工具。功能丰富，但对于简单任务可能显得过于复杂” 30 31 ‘ ‘ ‘

```
1
2 ## 需求分析
3 该产品应为一个基于Python3的图形界面应用程序，作为颜色计量器。它应允许
4 用户可以在屏幕上选择任何颜色，并显示相应的RGB值。RGB
5 值应实时更新，当颜色计器移动时。The interface should be
6 简单且用户友好。
7
8 ## 需求池
9 METAGPT: 多智能体协作框架的元编程
10 [
11     METAGPT: 多智能体协作框架的元编程 产品需求文档 "P0",
12     实现颜色选择功能 "P0",
13     显示所选颜色的RGB值 "P0",
14     实时更新色彩移动时的RGB值 "P0",
15     测试应用的准确性和性能 "P1"
16 ]
17
18 ## UI 设计草稿
19 用户界面应简洁且直观。其主要由一个颜色计量器组成。
20 可能是用户可以移动的屏幕放大区域。RGB值为
21 当前选择的颜色应突出显示，并实时更新在实时中
22 颜色计量器在移动。布局应保持简洁，并且整洁，重点突出
23 功能性和易用性。
24
25 ## 任何不清楚的地方
26 没有不清楚的地方。
```

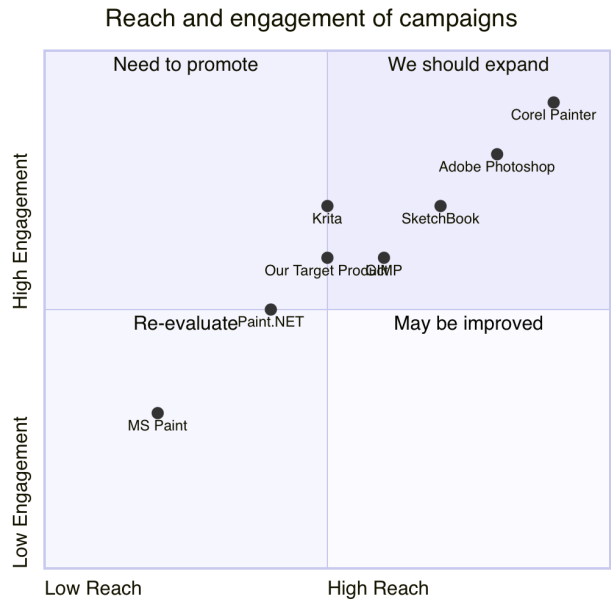
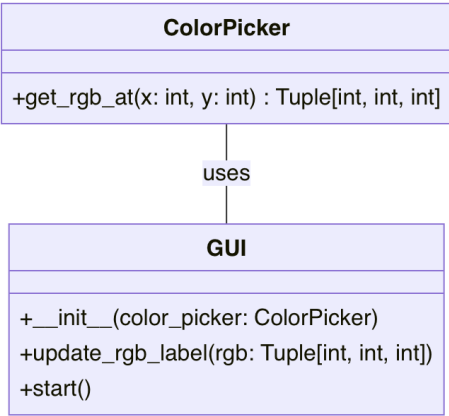


图7：由产品经理代理生成的“绘图应用”四象限图。

架构师根据PRD中的需求，架构师代理制定技术规范，包括系统架构图和接口定义。最初，架构师定义整体技术发展方向。随后，项目的架构设计完成，包括文件、类（图8）以及序列流程图（图12），然后将架构师的文档交给项目经理进行任务分配和执行。

系统设计

1 ## 实现方案 2 我们将使用Python的Tkinter库来创建颜色计的图形界面。Tkinter是一个标准的Python界面，用于Tk GUI工具包，且包含在大多数Python安装中。它使用简单，功能强大，足以满足我们的需求。3 4 颜色选择功能将通过PIL（Pillow）库实现，该库允许我们捕获屏幕的像素数据。通过移动鼠标，我们可以实时获取鼠标光标下像素的RGB值。5 6 RGB值将通过Tkinter的Label控件显示在GUI上。我们将使用Python的threading库在颜色计移动时实时更新RGB值。7 8 该应用程序将使用Python的unittest库进行准确性和性能测试。9 10 ## Python包名 11 'python 12 "color meter" 13 ' 14 15 ## 文件列表 16 'python 17 "main.py", 18 "color_picker.py", 19 "gui.py", 20 "tests.py" 21 22 23 ' 24 25 ## 任何不清楚的地方 26 需求对我来说是明确的。



Fig图8: 由architec生成的“绘图应用”数据结构和接口定义 agent. t

项目经理项目经理将项目拆分为任务列表。此外，每个代码文件根据其预期功能进行分析，然后作为单独的任务分配给工程师。

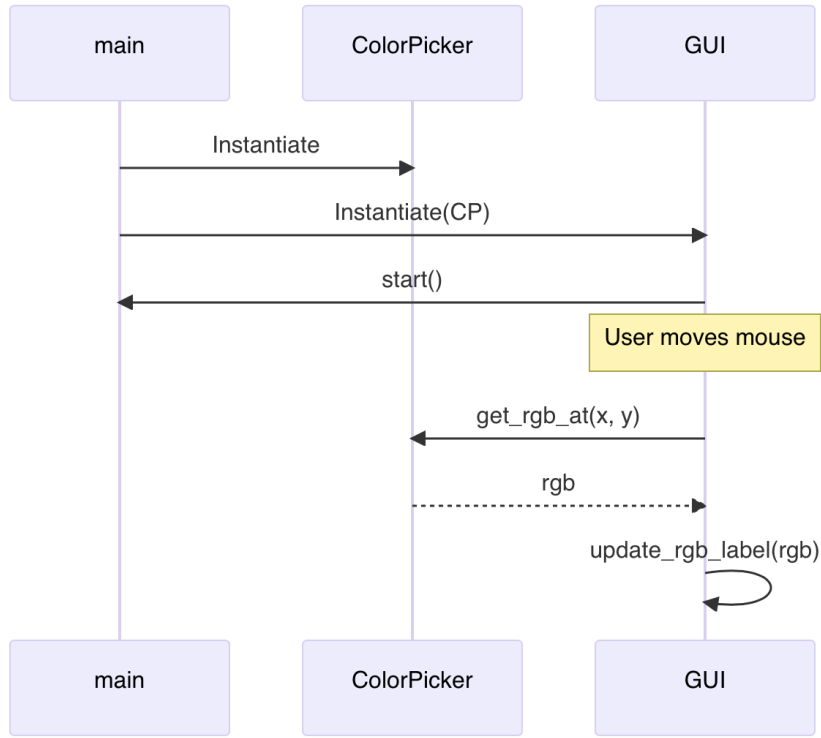


图9：“绘图应用”由architect 代理生成的程序调用流程

任务

```
## 所需的 Python 第三方包
2
3 任务
4 """
5 tkinter==8.6
6 Pillow .3.1
7 """
8 ...
9
10 ## 需要的其他语言第三方包
11 任务
12 """
13 其他语言不需要第三方包。
14 """
15 ...
16
17 ## 完整API规范
18 任务
19 """
20 本项目未使用任何API。
21 """
22 ...
23
24 ## 逻辑分析
25 任务
26 [
27     任务          任务
28     任务          实现绘图画布，响应鼠标事件，并使用所选的
29     工具与颜色。) 实现工具选择界面。
30     任务          实现颜色选择界面。
31     任务          实现文件保存与打开功能。
32 ]
33 ...
```

产品需求文档

```
## 任务列表
2  METAGPT: 多智能体协作框架的元编程
3  [
4      "main.py",
5      "canvas.py",
6      "tools.py",
7      "color_picker.py",
8      file_manager.py
9  ]
10  ```
11
## 共享知识
13  METAGPT: 多智能体协作框架的元编程
14  """
15  'Tkinter' 库用于创建应用程序的图形用户界面。它提供了各种控件
    ↳ 类似于本项目中使用的框架和画布。
16  Pillow 库用于处理图像文件。它在 'file_manager.py' 中被使用
    保存和打开图像文件。
17  main.py文件是应用程序的入口点。它初始化所有其他组件
    ↳ 并启动应用循环。
18  """
19  ```
20
21  ## 任何不清楚的地方
22  项目需求和实现方案已明确。然而，我们需要确保
    所有团队成员都熟悉 Tkinter 和 Pillow 库。如果没有，他们可能
    在开始开发之前，需要一些时间学习这些库。
```

工程师根据提供的文件结构和函数定义，工程师代理只需具备基本的开发技能即可完成开发任务。由于文件数量较多，此处仅展示一个自动生成的代码文件。

生成的代码（color picker.py）

```
1 import tkinter as tk
2 from tkinter import colorchooser
3 class ColorPicker:
4     def __init__(self, root: tk.Tk, color: str = 'black'):
5         self.frame = tk.Frame(root)
6         self.color = 颜色
7
8         self.color_按钮 = tk.Button(self.frame, text=' 颜色 ', command=self.select_颜色)
9         self.color_按钮.pack(fill='x')
10
11     def select_颜色(self):
12         color = colorchooser.askcolor()[1]
13         if color is not None:
14             self.color = 颜色
15
16     def pack(self, **kwargs):
17         self.frame.pack(**kwargs)
18
19     def get_color(self):
20         return self.color
```

QA Engineer 在收到工程师的代码输出后，QA工程师会生成单元测试代码并进行审查，以识别并修复潜在的缺陷，确保软件的高质量。

生成的单元测试（test.py）

```
1 import unittest
2 import tkinter as tk
3 # 你可能想将 ColorPicker 类移动到另一个文件，比如 colorpicker.py
4 # 来自颜色_picker 导入 ColorPicker
5
6 class TestColorPicker(unittest.TestCase):
7
8     def setUp(self):
9         self.root = tk.Tk()
10        self.cp = ColorPicker(self.root)
11
12    def tearDown(self):
13        # 这对于在测试后关闭 tk 窗口非常重要
14        self.root.destroy()
15
16    def test_initial_color(self):
17        # 初始化时测试默认颜色
18        self.assertEqual(self.cp.get_color(), ' black' )
19
20    def _set_and_get_color(self):
21        # 测试设置新颜色并获取它
22        new_color = '#ffffff' # 白色
23        self.cp.color = 新_颜色
24        self.assertEqual(self.cp.get_color(), new_color)
25
26
27 如果 __name__ == '__main__':
28    unittest.main()
```

Output 最终，如图 10 所示，MetaGPT 生成了一个名为“Drawing App”的功能性应用程序。

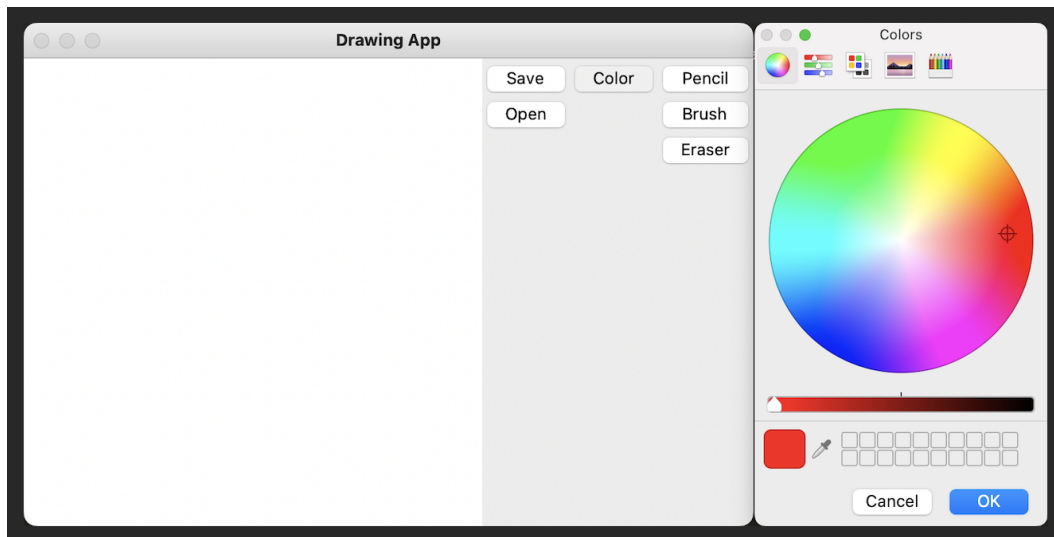


图10: 由MetaGPT生成的“绘图应用”

C 实验

C.1 软件开发数据集的详细信息

The SoftwareDev 数据集包含70个多样化的软件开发任务。表8显示了数据集中11个任务的名称和详细提示。请注意，列出的前七个任务用于本文的主要实验。

C.2 额外结果

MetaGPT的定量结果如表所示⁴，MetaGPT的平均得分为3.9，超过了ChatDev的2.1^{赵等人(2023)}，该方法基于聊天链。比较包括AutoGPT在内的通用智能算法的得分^{Torantulino等人(2023)}，它们的得分均为1.0，未能生成可执行代码。我们观察到，生成的代码通常较短，缺乏全面的逻辑，且在处理跨文件依赖关系时常常失败。虽然像 AutoGPT (^{Torantulino 等人, 2023})、Langchain (^{Chase, 2022})、以及 Agent-Verse (^{Chen 等人, 2023}) 展示出强大的通用问题解决能力，它们缺乏开发复杂系统的一个关键要素：系统性地拆解需求。相反，MetaGPT 通过专业的分工和 SOPs 工作流程，简化了将抽象需求转化为详细类和函数设计的过程。与 Chat-Dev (^{Zhao 等人, 2023}) 相比，MetaGPT 的结构化信息传递和反馈机制不仅减少了信息丢失，还提升了代码的执行效率。

MetaGPT在无可执行反馈情况下的定量结果表⁹展示了在SoftwareDev数据集的11个任务中，使用GPT-4 32K的MetaGPT的性能。最后一行还显示了在所有70个任务中的平均性能。请注意，此处使用的MetaGPT版本为基础版本，未集成可执行反馈机制。

MetaGPT：多智能体协作框架的元编程不同大型语言模型（LLMs）下MetaGPT的定量结果为了验证MetaGPT在不同LLMs上的性能，我们随机选择了5个软件开发任务，并使用GPT-3.5和Deepseek Coder 33B作为后端进行实验。正如表格⁵所示，结果表明，虽然MetaGPT可以使用这些LLMs完成任务，但采用GPT-4作为后端的性能更优越。

<https://deepseekcoder.github.io>

Table4: 可执行性比较。可执行性评分采用从“1”到“4”的等级系统。得分为“1”表示完全失败，“2”表示可执行代码，“3”代表基本满足预期工作流程，而“4”则表示与预期完全匹配。

Task	AutoGPT	LangChain	AgentVerse	ChatDev	MetaGPT
Flappy Bird	1	1	1	2	3
坦克战斗游戏	1	1	1	2	4
2048 游戏	1	1	1	1	4
贪吃蛇游戏	1	1	1	3	4
砖块破坏者游戏	1	1	1	1	4
Excel数据处理	1	1	1	4	4
CRUD 管理	1	1	1	2	4
平均分数	1.0	1.0	1.0	2.1	3.9

Table 5: MetaGPT在SoftwareDev上的性能，使用不同的大模型作为代理后端

模型	开源时间(/秒)
MetaGPT (配备 GPT-3.5)	✗ 75.18 161.6 2.8 2.4
MetaGPT (配备 GPT-4)	✗ 552.94 178.2 3.8 1.2
MetaGPT (配备Deepseek Coder 33B)	✓ 1186.20 120.2 1.4 2.6

指令级别的影响（高级 对比 详细指令）人类初始输入水平的变化是否会显著影响性能结果？例如：

- High-level prompt: 创建一个砖块破坏者游戏。
2. Detailed prompt: 创建一个打砖块游戏。在打砖块游戏中，玩家通常控制屏幕底部的挡板，将球弹向砖墙。目标是通过击打砖块用球将所有砖块打碎。

为探讨此方面，进行了额外的实验：我们从 SoftwareDev 中选择了 5 个任务，并为其构建了详细的提示。以下是实验结果：

表6: 指令级别的影响可执行性采用从“1”到“4”的评分系统。得分为“1”表示完全失败，“2”表示可运行代码，“3”代表基本符合预期的工作流程，“4”则表示与预期完全匹配。

模型	#词时间
高级水平	13.2 552.9 28384.2 178.2 3.8 163.8 1.2
METAGPT: 用于多智能体协作框架的元编程 C.2 额外结果	42.2 567.8 29657.0 257.0 4.0 118.0 1.6

我们观察到：详细提示能够带来更优的软件项目，生产率比率更低，这是由于需求和功能更为明确，而简单输入仍然可以利用MetaGPT生成足够优质的软件，其可执行性评分为3.8，与详细提示场景相当。（注意，生产率 = Token使用量 / 总代码行数。该比率越低越好。）

METAGPT: 多智能体协作框架的元编程GPT变体在HumanEval基准测试中的表现我们采用GPT-4在HumanEval中的67%得分（OpenAI, 2023）作为基线，认可其在HumanEval基准中的表现。我们在不同条件下对GPT-4（gpt-4-0613）和GPT-3.5 Turbo（gpt-3.5-turbo-0613）进行了五次扩展实验，以评估性能。(A) 我们直接使用OpenAI API，输入HumanEval中的提示语。(B) 我们调用OpenAI API，并在响应中使用正则表达式解析代码。(C) 我们添加了额外的系统提示，然后调用OpenAI API。提示内容为“你是一个只用Python代码回应的AI，不用英语。你将

METAGPT: 多智能体协作框架的元编程

表7: GPT模型在HumanEval上的表现 使用gpt-4-0613和gpt-3.5-turbo-0613在不同设置下进行了五次实验。

设置	模型	1	2	3	4	5	Avg.	Std.
A	METAGPT: 多智能体协作框架的元编程 C.2 额外结果	0.732	0.707	0.732	0.713	0.738	0.724	0.013
A	METAGPT: 多智能体协作框架的元编程 C.2 额外结果	0.360	0.366	0.360	0.348	0.354	0.357	0.007
B	METAGPT: 多智能体协作框架的元编程 D 限制与伦理关切	0.787	0.811	0.817	0.829	0.817	0.812	0.016
B	METAGPT: 多智能体协作框架的元编程 D 限制与伦理关切	0.348	0.354	0.348	0.335	0.348	0.346	0.007
C	METAGPT: 多智能体协作框架的元编程 D 限制与伦理关切	0.805	0.805	0.817	0.793	0.780	0.800	0.014
C	METAGPT: 多智能体协作框架的元编程 D 限制与伦理关切	0.585	0.567	0.573	0.579	0.579	0.577	0.007

定性结果 图 11 和图 12 展示了架构师代理在设计复杂推荐系统方面的努力。这些图展示了系统界面设计的整体方案和程序调用流程。后者对于构建一个复杂的自动化系统至关重要。强调在开发自动化软件框架时，这一分工的重要性具有关键意义。

D 限制与伦理关切

D.1 限制

Systemside 目前，我们的系统尚不能完全满足特定场景的需求，例如用户界面和前端，因为我们尚未集成此类代理和多模态工具。此外，尽管在同类框架中生成的代码最多，但仍然难以满足实际应用中多样化和复杂的需求。

Human user side 用户的一项关键挑战是中断每个代理的运行过程，或为每个代理设置起始运行点（检查点）。

D.2 伦理问题

失业与技能过时 MetaGPT 使更多人能够用自然语言进行编程，从而降低工程师入门的难度。多年来，编程语言已从打孔卡片发展到汇编语言、C、Java、Python，以及如今的自然语言。因此，人类在编程方面变得更加熟练，推动了与编程相关职位的需求增长。此外，使用自然语言进行编程可能提供显著更低的学习曲线，使得编程对更广泛的受众更为易及。

Transparency and Accountability MetaGPT 是一个开源框架，促进多个代理之间通过自然语言进行交互式通信。人类可以以最高的控制级别发起、观察和停止运行。它提供自然语言的实时解释和操作，显示在屏幕和日志中，确保透明度。MetaGPT 增强了“自然语言编程”能力，人类工程师是用户并对结果负责。

隐私与数据安全 MetaGPT在本地运行，确保用户数据的隐私与安全。它不收集用户数据。对于与第三方大型语言模型（LLMs）如OpenAI的交互，建议用户参考相关隐私政策（例如OpenAI隐私政策）。然而，我们提供开源大型语言模型作为后端的选择。

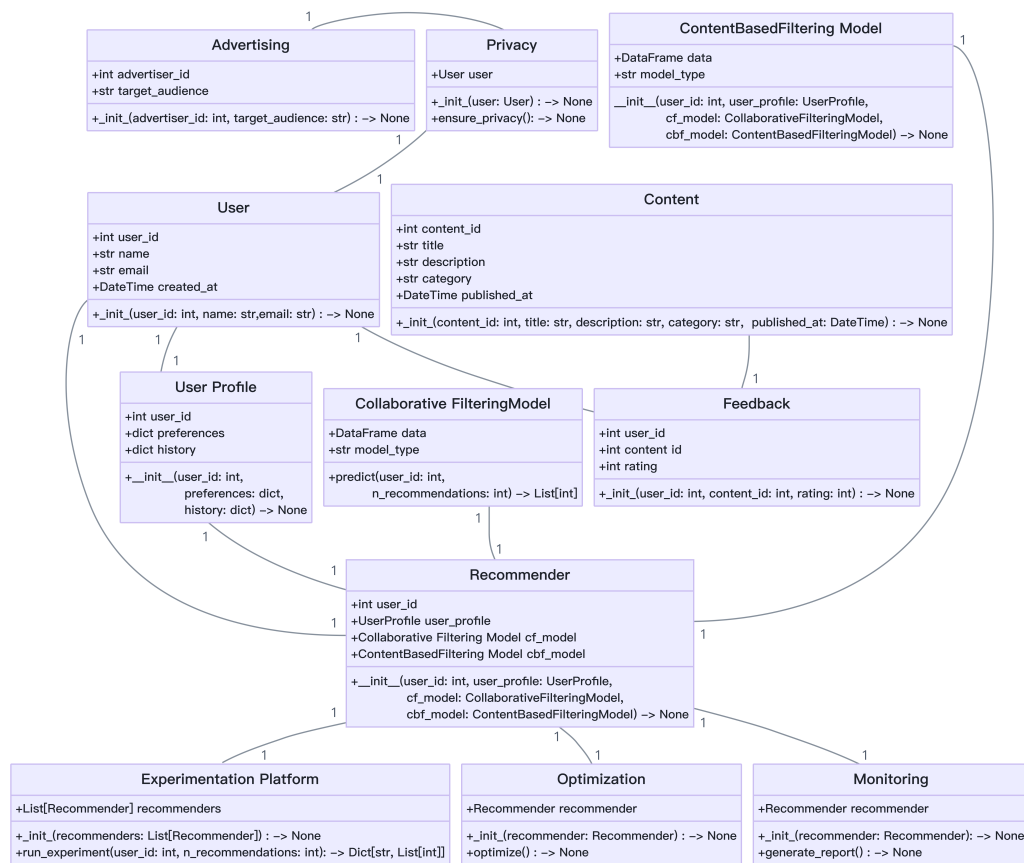


图11：“推荐引擎开发”系统界面设计由架构师代理生成（放大以获得更好视图）。

E 更多讨论

E.1 根深蒂固的挑战

MetaGPT 也通过其独特的设计缓解或解决了这些挑战：

高效利用上下文存在两个子挑战。首先，准确展开简短的自然语言描述以消除歧义。其次，在长篇上下文中保持信息的有效性，使大型语言模型（LLMs）能够专注于相关数据而不被干扰。

减少幻觉 使用大规模语言模型（LLMs）生成完整的软件程序时，面临代码幻觉问题——包括功能实现不完整、依赖缺失以及潜在未被发现的漏洞，这些问题可能更为严重。由于任务定义模糊，LLMs在软件生成方面常常表现不佳。专注于需求分析、包选择等细粒度任务，有助于引导思维，而这是LLMs在解决宽泛任务时所缺乏的。

E.2 信息过载

在MetaGPT中，我们采用全局消息池和订阅机制来应对“信息过载”，即接收过多或无关信息的问题。此问题依赖于具体应用场景。MetaGPT利用消息池来简化通信流程，确保效率。此外，订阅机制可以过滤掉无关的上下文，提升信息的相关性和实用性。这一设计在软

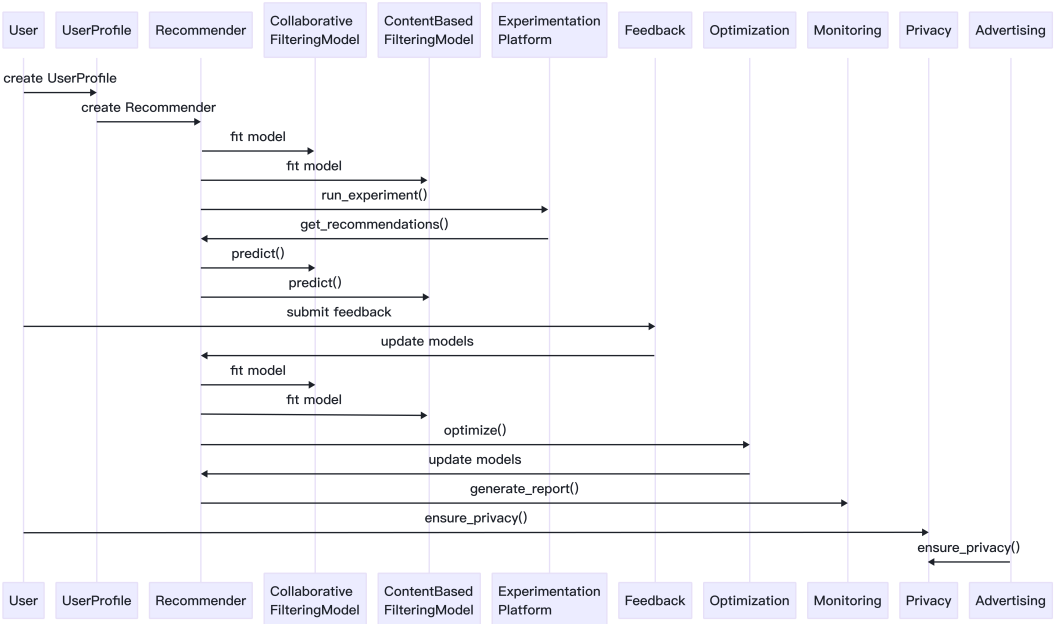


图12：“推荐引擎开发”程序调用流程，由架构师代理生成（放大以获得更佳视图）。

在需要高效沟通的硬件设计场景和标准操作程序（SOPs）中。

表8: SoftwareDev数据集示例

任务编号 任务		METAGPT: 多智能体协作框架的元编程 E.2 信息过载
0	贪吃蛇游戏	创建一个贪吃蛇游戏。
1	砖块破坏者游戏	创建一个砖块破坏者游戏。
2	2048 游戏	为网页创建一个2048游戏。
3	Flappy Bird 游戏	请为 Flappy Bird 编写 p5.js 代码，控制一只黄色鸟连续-连续在一系列绿色管道之间飞行。每次你<style id='l'>...</style>鸟振翅时左键点击鼠标。如果它掉到地面或碰到管道，你就输了。游戏无限进行，直到你失败为止；你越往前走，得分越高。
4	坦克战斗游戏	创建一款坦克战斗游戏
5	Excel数据处理	基于Streamlit和Pandas编写Excel数据处理程序。屏幕首先显示一个Excel文件上传按钮。文件上传后，使用 pandas 显示其数据内容。程序需简洁，易于维护，且不过度设计。它使用 streamlit 处理网页屏幕显示，pandas 足以处理Excel读取和显示。 E.2 信息过载
6	CRUD管理	基于增删改查的管理程序编写 客户业务实体的认知与查询处理。客户需要保存此信息：姓名、出生日期、年龄、性别和电话。数据存储在 client.db 中，并且存在对 customer 表是否存在的判断——如果不存在，则需要先创建。查询是通过名称进行的；相同用于删除。该程序要求简洁、易于维护，并且不过度设计。该界面通过 streamlit 和 sqlite 实现——无需引入其他附加包。
7	音乐转录器	开发一个程序，将乐谱转录为数字格式；提供从音频中提取无误的转录符号化乐谱信息 通过涉及音高和时间切片的信号处理，然后进行训练神经网络运行于 Onset Detected CWT 转换标度图为色度图用递归神经网络解码的图形。
8	自定义新闻稿 创建自定义新闻稿；	开发一个Python脚本，用于提取相关内容。来自外部渠道的公司新闻信息，例如社交媒体媒体；提取更新间隔数据库以获取最新变化。程序应创建具有可定制选项的新闻稿并导出文稿到PDFs、NYTimes API JSONs、带有内部互联样式的媒体格式固定字符长度元数据
9	五子棋游戏	使用Python实现五子棋游戏，集成人工智能对手难度等级各异。
10	天气仪表盘	创建一个Python程序以开发交互式天气仪表盘。

Table 9: **Additional results of pure MetaGPT w/o feedback on SoftwareDev.** Averages (Avg.) of 70 tasks are calculated and 10 randomly selected tasks are included. ‘#’ denotes ‘The number of’, while ‘ID’ is ‘Task ID’.

ID	Code statistics			Doc statistics			Cost statistics			Cost of revision	Code executability	
	#code files	#lines of code	#lines per code file	#doc files	#lines of doc	#lines per doc file	#prompt tokens	#completion tokens	time costs			money costs
0	5.00	196.00	39.20	3.00	210.00	70.00	24087.00	6157.00	582.04	\$ 1.09	1. TypeError	4
1	6.00	191.00	31.83	3.00	230.00	76.67	32517.00	6238.00	566.30	\$ 1.35	1. TypeError	4
2	3.00	198.00	66.00	3.00	235.00	78.33	21934.00	6316.00	553.11	\$ 1.04	1. lack @app.route('/')	3
3	5.00	164	32.80	3.00	202.00	67.33	22951.00	5312.00	481.34	\$ 1.01	1. PNG file missing 2. Compile bug fixes	2
4	6.00	203.00	33.83	3.00	210.00	70.00	30087.00	6567.00	599.58	\$ 1.30	1. PNG file missing 2. Com-pile bug fixes 3. pygame.surface not initialize	3
5	6.00	219.00	36.50	3.00	294.00	96.00	35590.00	7336.00	585.10	\$ 1.51	1. dependency error 2. ModuleNotFoundError	4
6	4.00	73.00	18.25	3.00	261.00	87.00	25673.00	5832.00	398.83	\$ 0.90	0	4
7	4.00	316.00	79.00	3.00	332.00	110.67	29139.00	7104.00	435.83	\$ 0.92	0	4
8	5.00	215.00	43.00	3.00	301.00	100.33	29372.00	6499.00	621.73	\$ 1.27	1. tensorflow version error 2. model training method not implement	2
9	5.00	215.00	43.00	3.00	270.00	90.00	24799.00	5734.00	550.88	\$ 1.27	1. dependency error 2. URL 403 error	3
10	3.00	93.00	31.00	3.00	254.00	84.67	24109.00	5363.00	438.50	\$ 0.92	1. dependency error 2. missing main func.	4
Avg.	4.71	191.57	42.98	3.00	240.00	80.00	26626.86	6218.00	516.71	\$ 1.12	0.51 (only consider item scored 2, 3 or 4)	3.36